Lesson 1: Meet Thonny & Python 🐍

> **Goal:** Open Thonny, write your first Python program, and understand what a "program" is.

---

1. What is Python?

Python is a **programming language** — a way to talk to the computer.

In Scratch, you drag colorful blocks to make programs. In Python, you **type words** to make programs.

Scratch	Python
Drag blocks	Type code
Click green flag to run	Click Run button to run
"say" block	`print()`

Python is used by real programmers all over the world to build games, websites, robots, and more!

---

2. Meet Thonny — Your Code Editor

Thonny is a special program where you write and run Python code.

How to open Thonny:

1. Find the **Thonny** icon on your desktop or Start menu.
2. Double-click to open it.

Thonny has 3 important parts:

```
┌──────────────────────────────┐
│  Menu bar (File, Run, ...)   │
├──────────────────────────────┤
│                              │
│   CODE EDITOR                │
│   (you type code here)       │
│                              │
├──────────────────────────────┤
│   SHELL                      │
```

```
│   (results show here)        │
│   >>>                        │
└──────────────────────────────┘
```

- **Code Editor** (top) — where you type your program.
- **Shell** (bottom) — where you see the output (results).
- **Run button ▶** — click to run your program (or press **F5**).

---

3. Your First Program: Hello World!

In the **Code Editor**, type this:

```
print("Hello, World!")
```

Then click **Run ▶** (or press F5).

You will see this in the **Shell**:

```
Hello, World!
```

🎉 **Congratulations!** You just wrote your first Python program!

How does it work?

- `print()` is a **function** — it tells the computer to show something on the screen.
- `"Hello, World!"` is a **string** — text inside quotes.
- The computer reads your code and follows your instructions.

> 💡 **Scratch connection:** `print()` is like the **"say"** block in Scratch!

---

4. Print More Things

You can print anything you want! Try these:

```
# Print your name
print("My name is Python")

# Print a number
print(123)
```

```
# Print a math result
print(5 + 3)
```

Output:

```
My name is Python
123
8
```

Important rules:

Rule	Example	What happens
Text must be in quotes	`print("hello")`	✅ Shows: hello
Numbers don't need quotes	`print(42)`	✅ Shows: 42
Forgot quotes for text	`print(hello)`	❌ Error!

---

5. Comments — Notes for Humans

A **comment** starts with `#`. Python ignores comments.

```
# This is a comment. Python skips this line.
print("This line runs!")  # This comment is at the end
```

Comments help you remember what your code does. Write comments in English!

---

6. Print Multiple Lines

You can use many `print()` to show many lines:

```
print("Line 1")
print("Line 2")
print("Line 3")
```

Output:

```
Line 1
Line 2
Line 3
```

Print an empty line:

```
print("Hello")
print()          # This prints an empty line
print("World")
```

Output:

```
Hello

World
```

---

7. Common Errors

When you make a mistake, Python shows an **error message**. Don't worry — errors help you learn!

Mistake	Code	Error
Forgot quotes	`print(hello)`	`NameError`
Forgot parentheses	`print "hello"`	`SyntaxError`
Wrong spelling	`prnt("hello")`	`NameError`

> 💡 **Tip:** Read the error message carefully. It tells you what went wrong!

---

8. Vocabulary

English	Vietnamese	Meaning
program	chương trình	A set of instructions for the computer
code	mã lệnh	The text you write in a programming language
run	chạy	Tell the computer to follow your instructions
print	in ra	Show something on the screen
string	chuỗi	Text inside quotes
comment	ghi chú	A note for humans, Python ignores it
error	lỗi	A mistake in your code

English	Vietnamese	Meaning
shell	cửa sổ lệnh	Where you see the output
function	hàm	A command that does something (like `print`)

---

9. Summary

✅ Python is a text-based programming language. ✅ Thonny is the editor where you write and run Python code. ✅ `print()` shows text or numbers on the screen. ✅ Text (strings) must be inside quotes: `"hello"`. ✅ Comments start with `#` and are ignored by Python. ✅ Errors are normal — read the message and fix your code!

---

10. Homework

🟢 Easy

Exercise E1. Say Hello

Write a program that prints the word `Hello` on the screen.

- **Input:** None.
- **Output:** Print exactly one line: `Hello`

Example:

```
Hello
```

Exercise E2. Your Name

Write a program that prints your name. For example, if your name is "An", print `An`.

- **Input:** None.
- **Output:** Print exactly one line with your name.

Example:

```
An
```

Exercise E3. Three Lines

Write a program that prints these 3 lines exactly:

- **Input:** None.
- **Output:**

```
I like Python
Python is fun
I am a coder
```

Exercise E4. A Number

Write a program that prints the number `2025`.

- **Input:** None.
- **Output:** Print exactly: `2025`

Example:

```
2025
```

Exercise E5. Math Result

Write a program that prints the result of `10 + 25`.

- **Input:** None.
- **Output:** Print the result of 10 + 25.

Example:

```
35
```

Exercise E6. Empty Line

Write a program that prints:

- Line 1: `Hello`
- Line 2: (empty)

- Line 3: `World`
- **Input:** None.
- **Output:**

```
Hello

World
```

Exercise E7. Star Box

Write a program that prints this pattern:

- **Input:** None.
- **Output:**

```
*****
*   *
*   *
*****
```

🟡 Medium

Exercise M1. Greeting Card

Write a program that prints a greeting card like this:

- **Input:** None.
- **Output:**

```
+------------------+
|  Happy Birthday! |
|    Dear An       |
+------------------+
```

Exercise M2. Math Table

Write a program that prints the results of these calculations:

- **Input:** None.

- **Output:**

```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
```

(Hint: you can write `print("10 + 5 =", 10 + 5)`)

Exercise M3. My Info

Write a program that prints 4 lines about yourself:

- **Input:** None.
- **Output:** (example)

```
Name: An
Age: 8
School: ABC Primary School
Hobby: coding
```

Exercise M4. Triangle

Write a program that prints this triangle:

- **Input:** None.
- **Output:**

```
*
**
***
****
*****
```

Exercise M5. Number Pyramid

Write a program that prints this number pattern:

- **Input:** None.
- **Output:**

```
1
12
123
1234
12345
```

Exercise M6. House

Write a program that prints a simple house using text characters:

- **Input:** None.
- **Output:**

```
   /\
  /  \
 /    \
+------+
|      |
|  []  |
+------+
```

Exercise M7. Multiplication

Write a program that prints the result of `123 * 456` using `print()`.

- **Input:** None.
- **Output:** Print the result of 123 × 456.

Example:

```
56088
```

🔴 Hard

Exercise H1. Planting Trees

Inspired by competitive programming for kids.

The mayor wants to plant trees along a road. The road is **N** meters long. A tree is planted every **10 meters**, starting at position 10 m (not at position 0 m).

Write a program that calculates how many trees are needed.

- **Input:** One integer N (10 ≤ N ≤ 1000) — the length of the road in meters.
- **Output:** One integer — the total number of trees.

Example:

Input	Output
100	10
35	3
10	1

Explanation: For a 100 m road, trees are at 10, 20, 30, ..., 100 → 10 trees. For a 35 m road, trees are at 10, 20, 30 → 3 trees (no tree at 35 because it's not a multiple of 10).

(Hint: Think about division. You will learn `input()` in Lesson 4. For now, just set `n = 100` in your code and change it to test.)

Exercise H2. Candy Sharing

An has **N** candies. She wants to share equally among **3** friends. Each friend gets the same number of candies. The leftover candies stay with An.

Write a program that prints how many candies each friend gets, and how many are left.

- **Input:** One integer N (1 ≤ N ≤ 1000) — the number of candies.
- **Output:** Two lines:
  - Line 1: the number of candies each friend gets.
  - Line 2: the number of leftover candies.

Example:

Input	Output
10	3
	1
7	2
	1

Explanation: 10 ÷ 3 = 3 remainder 1. Each friend gets 3, An keeps 1.

(Hint: Use `//` for division and `%` for remainder. Set `n = 10` in your code.)

Exercise H3. Fence Posts

A farmer wants to build a fence around a **square** garden. Each side of the garden is **S** meters long. He places a fence post every **1 meter** along each side.

How many fence posts does he need in total? (Be careful: don't count corner posts twice!)

- **Input:** One integer S (1 ≤ S ≤ 100) — the side length of the square garden.
- **Output:** One integer — the total number of fence posts.

Example:

Input	Output
4	16
1	4
10	40

Explanation: For S = 4: each side has 5 posts (at 0m, 1m, 2m, 3m, 4m), but 4 corners are shared. Total = 4 × 5 - 4 = 16. Or think: 4 × S = 4 × 4 = 16.

(Hint: Each side has S+1 posts, but 4 corners are shared between 2 sides. Total = 4 × (S+1) - 4 = 4 × S.)

Exercise H4. Book Pages

A book has **N** pages. Each page is numbered from 1 to N. How many **digits** are used in total to number all the pages?

- **Input:** One integer N (1 ≤ N ≤ 100) — the number of pages.
- **Output:** One integer — the total number of digits used.

Example:

Input	Output
5	5
11	13

Input	Output
100	192

Explanation: For N = 11: pages 1-9 use 1 digit each (9 digits), pages 10-11 use 2 digits each (4 digits). Total = 9 + 4 = 13.

(Hint: Pages 1-9 use 1 digit each. Pages 10-99 use 2 digits each. Page 100 uses 3 digits. You will learn loops later — for now, try to find a formula!)

Exercise H5. Clock

A digital clock shows time in the format `H:MM`. Given the total number of **minutes** since midnight, print the time.

- **Input:** One integer M (0 ≤ M ≤ 1439) — minutes since midnight.
- **Output:** The time in format `H:MM` (hours without leading zero, minutes always 2 digits).

Example:

Input	Output
0	0:00
150	2:30
1000	16:40

Explanation: 150 minutes = 2 hours and 30 minutes → 2:30.

(Hint: Use `//` to find hours and `%` to find minutes. For now, set `m = 150` in your code.)